
Efficient Python

Lecture 01

CERN
STEAM Academy

06 July 2026

# Vector, Matrix, and Tensor Thinking

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland

AGH

# Outline

Outline of the course

A very quick recap

Vector, Matrix, and Tensor

Further reading

Section 1

# Outline of the course

# Lectures

- **01 – Vector, Matrix, and Tensor Thinking**
  This lecture introduces vectors, matrices, and tensors as computational array objects and shows how mathematical notation translates into shape-, axis-, and indexing-aware Python computation.
- **02 – From Arrays to Visual Numerical Experiment**
  This lecture moves from array concepts to practical numerical experimentation with NumPy and Matplotlib, emphasizing construction, transformation, visualization, and visual sanity checks of numerical data.
- **03 – Numerical Foundations for Machine Learning**
  This lecture introduces the numerical structures underlying machine learning workflows, including data arrays, model computations, losses, gradients, scaling, stability, and iterative training loops.
- **04 – Idiomatic Scientific Python**
  This lecture introduces idiomatic scientific Python practices that improve the reliability, readability, testability, and maintainability of numerical code through explicit interfaces, controlled state, lightweight structure, and reusable data-flow patterns.

# Lectures

- **05 – Portable Data Pipelines and Labelled Analysis**
  This lecture introduces portable, reproducible data analysis pipelines using robust file handling, project organization, and pandas-based labelled, vectorized workflows for cleaning, transforming, aggregating, and visualizing scientific data.
- **06 – Measurement-Guided Optimization**
  This lecture introduces a measurement-guided approach to optimization, covering representative benchmarking, runtime profiling, memory inspection, and debugging practices that support disciplined performance improvements.
- **07 – Acceleration and Scalable Execution**
  This lecture presents practical strategies for accelerating scientific Python code after measurement, including algorithmic improvement, memory-aware vectorization, JIT compilation, concurrency, parallel execution, and trade-off-based optimization decisions.

# Recommended reading


- Harris, C.R., Millman, K.J., van der Walt, S.J. *et al.* Array programming with NumPy. *Nature* **585**, 357–362 (2020). DOI: `https://doi.org/10.1038/s41586-020-2649-2`
- VanderPlas, J. *Python Data Science Handbook: Essential Tools for Working with Data*. 2nd ed. O'Reilly Media, 2023. Online version: `https://jakevdp.github.io/PythonDataScienceHandbook/` Publisher page: `https://www.oreilly.com/library/view/python-data-science/9781098121211/`
- McKinney, W. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3rd ed. O'Reilly Media, 2022. Online version: `https://wesmckinney.com/book/` Publisher page: `https://www.oreilly.com/library/view/python-for-data/9781098104023/`
- Gorelick, M., Ozsvald, I. *High Performance Python: Practical Performant Programming for Humans*. 3rd ed. O'Reilly Media, 2025. Publisher page: `https://www.oreilly.com/library/view/high-performance-python/9781098165956/`

# Recommended reading


- Scopatz, A., Huff, K.D. *Effective Computation in Physics: Field Guide to Research with Python*. O'Reilly Media, 2015. ISBN: 9781491901564. Publisher page: `https://www.oreilly.com/library/view/effective-computation-in/9781491901564/`
- Lam, S.K., Pitrou, A., Seibert, S. Numba: A LLVM-based Python JIT compiler. In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (LLVM '15), 1–6 (2015). DOI: `https://doi.org/10.1145/2833157.2833162` Project documentation: `https://numba.pydata.org/`
- NumPy Developers; pandas Development Team; Python Software Foundation. Official documentation for NumPy, pandas, and Python profiling/debugging tools. NumPy User Guide: `https://numpy.org/doc/stable/user/` pandas User Guide: `https://pandas.pydata.org/docs/user_guide/` Python Profilers: `https://docs.python.org/3/library/profile.html` `tracemalloc`: `https://docs.python.org/3/library/tracemalloc.html` `pdb`: `https://docs.python.org/3/library/pdb.html`

Section 2

# A very quick recap

# Names, Values, and Numerical Expressions

In Python, a variable name is bound to a value.

```
x = 2.0
y = x
x = x + 1.0
```

Common numerical types:

```
n = 3           # int
x = 3.14        # float
z = 2 + 1j      # complex
flag = x > 0    # bool
```

Mathematical expressions become executable expressions:

```
energy = 0.5 * mass * velocity**2
```

Key idea: Python variables are not symbolic variables; they are names referring to concrete runtime objects.

# Ordered Containers, Indexing, and Slicing

Python lists and tuples store ordered collections of values.

```
v = [10, 20, 30, 40]
point = (1.0, 2.0, 3.0)
```

Indexing is zero-based:

```
v[0]    # first element: 10
v[2]    # third element: 30
v[-1]   # last element: 40
```

Slicing selects ranges:

```
v[1:3]    # [20, 30]
v[:2]     # [10, 20]
v[::2]    # [10, 30]
```

Nested lists can represent simple table-like data:

```
M = [[1, 2, 3],
     [4, 5, 6]]
M[1][2]   # 6
```

# Iteration, Transformation, and Reduction

Loops express repeated computation:

```
values = [1, 2, 3, 4]

total = 0
for x in values:
    total = total + x
```

`range` is commonly used for index-based iteration:

```
for i in range(len(values)):
    print(i, values[i])
```

`enumerate` gives both index and value:

```
for i, x in enumerate(values):
    print(i, x)
```

Comprehensions build transformed collections:

```
squares = [x**2 for x in values]
```

# Functions and Modules

Functions package a computation into a reusable form.

```
def mean(values):
    total = 0.0
    for x in values:
        total = total + x
    return total / len(values)
```

A function should make inputs and outputs clear:

```
data = [1.0, 2.0, 3.0]
m = mean(data)
```

Modules provide organized collections of tools:

```
import math

r = math.sqrt(2.0)
angle = math.pi / 4
```

Section 3

# Vector, Matrix, and Tensor

# Why array thinking comes first

Scientific Python is most effective when mathematical structure is translated into array structure. Before discussing performance tricks, compilers, parallelism, or GPUs, we need a precise way of representing vectors, matrices, fields, operators, parameters, and ensembles as numerical arrays.

The central idea of this lecture is that an array is not merely a container for numbers. It carries information about dimensionality, shape, axes, data type, memory layout, and computational cost. Efficient scientific programming starts when these features become part of how we formulate the problem.

```
import numpy as np

x = np.linspace(0.0, 1.0, 1000)
u = np.sin(2*np.pi*x)
```

This is already a mathematical statement: a continuous function has been sampled on a grid, producing a finite-dimensional numerical object.

# Mathematical objects versus computational objects

In mathematics, symbols often hide implementation details. A vector $\boldsymbol{x}$, a matrix $\boldsymbol{A}$, or a tensor $\boldsymbol{T}$ may be manipulated abstractly without specifying storage, precision, or indexing conventions. In numerical code, these details become unavoidable.

A computational vector has a finite length, a data type, an order of entries, and a representation in memory. A matrix has not only dimensions $m \times n$, but also a chosen storage layout. A sampled field has grid ordering, boundary conventions, and units associated with axes.

```
x = np.array([1.0, 2.0, 3.0])

x.shape      # (3,)
x.ndim       # 1
x.dtype      # dtype('float64')
x.size       # 3
```

The mathematical object is idealized. The computational object is finite, approximate, indexed, and stored.

# Arrays as discretized physical objects

Many physical objects become arrays after discretization. A particle state may become a vector. A linearized operator may become a matrix. A scalar field $u(x, t)$ may become a two-dimensional array. A field $u(t, x, y)$ becomes a three-dimensional array. An ensemble of simulations adds one more axis.

```
state = np.array([x0, v0])           # one particle in 1D
trajectory = np.zeros((nt, 2))       # time series of (x, v)

field_1d = np.zeros((nt, nx))        # u(t, x)
field_2d = np.zeros((nt, nx, ny))    # u(t, x, y)

ensemble = np.zeros((nrun, nt, nx)) # many simulations
```

The additional axes are not accidental. They represent physical, numerical, or statistical structure. A well-designed array shape makes later operations almost self-explanatory.

# Shape is part of the mathematical model

The shape of an array should be treated as part of the model specification. If we simulate $N$ particles in $d$ spatial dimensions, then `positions.shape == (N, d)` is not just an implementation choice. It expresses that `axis 0` enumerates particles and `axis 1` enumerates spatial components.

```
N = 1000
d = 3

positions = np.zeros((N, d))
velocities = np.zeros((N, d))
masses = np.ones(N)

# positions[i, alpha] = alpha-th coordinate of particle i
```

A different shape, for example $(d, N)$, may be equally valid, but it changes how we write reductions, broadcasts, and linear algebra operations. Therefore, array shape should be chosen deliberately, not discovered accidentally during debugging.

# Rank, dimensionality, and axes

In NumPy, the number of axes of an array is called its number of dimensions, available as `ndim`. This is closely related to what is often called the rank of an array in programming contexts. One should be careful, however, because “rank” in linear algebra may also mean the dimension of the image of a matrix.

```
scalar = np.array(3.14)
vector = np.zeros(5)
matrix = np.zeros((4, 5))
tensor_like = np.zeros((10, 4, 5))

scalar.ndim       # 0
vector.ndim       # 1
matrix.ndim       # 2
tensor_like.ndim  # 3
```

The axes of an array are ordered. This ordering affects indexing, broadcasting, memory access patterns, and the interpretation of operations such as `sum(axis=...)`.

# One-dimensional arrays are not automatically column vectors

In mathematical notation, we often distinguish between row vectors and column vectors. In NumPy, a one-dimensional array has shape `(n,)`; it is neither a row vector nor a column vector. It has only one axis.

```
x = np.array([1.0, 2.0, 3.0])

x.shape             # (3,)

x_col = x[:, None]
x_row = x[None, :]

x_col.shape         # (3, 1)
x_row.shape         # (1, 3)
```

This distinction becomes important when broadcasting, forming outer products, or multiplying matrices. Treating `(n,)`, `(n, 1)`, and `(1, n)` as interchangeable is one of the most common sources of shape bugs.

# Vectors as states, samples, and parameters

A one-dimensional array may represent many different mathematical objects. It may be a coordinate vector in phase space, a sampled function, a list of parameters, a frequency spectrum, or the flattened state of an ODE solver.

```
x_grid = np.linspace(-10.0, 10.0, nx)
u = np.tanh(x_grid)                  # sampled field

theta = np.array([alpha, beta, gamma])  # model parameters

y0 = np.array([x0, v0, a0, adot0])       # ODE initial state
```

The same computational shape `(n,)` may correspond to very different mathematical meanings. Good naming is therefore not cosmetic; it prevents confusion between coordinates, values, modes, parameters, and states.

# Matrices are not always linear operators

A two-dimensional array may represent a linear map, but it may also represent a data table, a scalar field on a two-dimensional grid, an image, a covariance matrix, or a kernel evaluated at pairs of points.

```
A = np.eye(4)               # linear operator
data = np.zeros((100, 5))   # observations × features
u = np.zeros((nx, ny))      # scalar field u(x, y)
K = np.zeros((nx, nx))      # kernel K(x_i, x_j)
```

The operation we apply must respect the interpretation. For a linear operator, `A @ x` may be meaningful. For a two-dimensional scalar field, `u @ x` usually has no physical meaning unless we have deliberately defined such an operation.

# Indexing as a bridge between notation and implementation

Index notation is often the most precise bridge between mathematics and array programming. If we write $u_{ij}$, we must decide which physical coordinate corresponds to $i$, which corresponds to $j$, and how this is stored.

```
u = np.zeros((nx, ny))

# u[i, j] represents u(x_i, y_j)
value = u[i, j]
```

In NumPy, indexing starts at zero. Slices are half-open: `u[1:5]` contains indices `1`, `2`, `3`, and `4`. These conventions are simple, but they matter in finite differences, boundary treatment, and grid-based algorithms.

```
interior = u[1:-1, 1:-1]
left_boundary = u[0, :]
right_boundary = u[-1, :]
```

# Grids and sampled fields

A continuous scalar field $u(x, y)$ becomes an array once we choose a grid. The grid itself is also represented by arrays. In physics-oriented code, `indexing="ij"` is often preferable because it preserves the convention $u[i, j] = u(x_i, y_j)$.

```
x = np.linspace(-5.0, 5.0, nx)
y = np.linspace(-3.0, 3.0, ny)

X, Y = np.meshgrid(x, y, indexing="ij")
u = np.exp(-(X**2 + Y**2))

u.shape      # (nx, ny)
X.shape      # (nx, ny)
Y.shape      # (nx, ny)
```

This representation makes formulas such as $u(x, y) = \exp[-(x^2 + y^2)]$ almost directly translatable into array code.

→ **Example 01 - 01**

# Elementwise operations and universal functions

Elementwise operations act independently on each entry of an array and usually preserve shape. They are natural for nonlinearities in differential equations, pointwise potentials, source terms, coordinate transformations, and filtering operations.

```
phi = np.linspace(-2.0, 2.0, 500)

V = 0.25 * (phi**2 - 1.0)**2
dV = phi * (phi**2 - 1.0)

s = np.sin(phi)
e = np.exp(-phi**2)
```

NumPy functions such as `np.sin`, `np.exp`, `np.sqrt`, and `np.tanh` are vectorized universal functions. They apply the scalar mathematical function to all entries of the array, but the loop over entries is performed in optimized compiled code rather than in Python.

# Broadcasting as controlled implicit expansion

Broadcasting allows NumPy to combine arrays with different but compatible shapes. It is one of the most important mechanisms for writing compact and efficient scientific code.

```
x = np.linspace(0.0, 1.0, nx)        # shape (nx,)
a = np.array([1.0, 2.0, 3.0])        # shape (3,)

values = a[:, None] * np.sin(2*np.pi*x[None, :])

values.shape                         # (3, nx)
```

Here, `a[:, None]` has shape $(3, 1)$ and `x[None, :]` has shape $(1, nx)$. The result has shape $(3, nx)$, representing three different amplitudes evaluated on the same spatial grid.

Broadcasting is powerful, but it must be intentional. A wrong singleton axis may silently produce an array with a plausible shape but incorrect meaning.

# Creating new axes deliberately

Adding new axes is a way to express outer constructions, parameter sweeps, pairwise distances, or grid-based functions. The operation `None` or `np.newaxis` does not copy data; it changes how the array participates in broadcasting.

```
x = np.linspace(-1.0, 1.0, nx)
y = np.linspace(-2.0, 2.0, ny)

R2 = x[:, None]**2 + y[None, :]**2
u = np.exp(-R2)

R2.shape    # (nx, ny)
```

This pattern appears in many scientific problems: constructing potentials $V(x, y)$, evaluating pairwise kernels $K(x_i, y_j)$, computing distances, or generating families of functions.

# Reductions: sums, means, norms, and integrals

A reduction collapses one or more axes. In mathematics, this corresponds to summation, averaging, integration, maximization, or taking a norm. In array programming, the key question is: which axes should disappear?

```
u = np.zeros((nt, nx, ny))

mass_t = np.sum(u, axis=(1, 2)) * dx * dy
max_t = np.max(u, axis=(1, 2))
mean_x_profile = np.mean(u, axis=2)
```

If `u.shape == (nt, nx, ny)`, then `mass_t.shape == (nt,)`: spatial axes have been reduced, while the time axis remains. This is exactly the discrete analogue of computing a spatial integral as a function of time.

# Keeping dimensions for later broadcasting

Sometimes we want to reduce an axis but preserve the number of dimensions. This is useful when the reduced quantity will be subtracted, divided, or otherwise combined with the original array.

```
u = np.random.rand(nt, nx, ny)

mean_space = u.mean(axis=(1, 2), keepdims=True)
u_centered = u - mean_space

mean_space.shape   # (nt, 1, 1)
u_centered.shape   # (nt, nx, ny)
```

The option `keepdims=True` preserves singleton axes, which makes broadcasting explicit and safe. Without it, the result would have shape $(nt,)$, which may not broadcast in the intended way against $(nt,\ nx,\ ny)$.

# Inner products and norms as reductions

An inner product combines elementwise multiplication with a reduction:

$$x \cdot y = \sum_i x_i y_i.$$

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([0.5, 0.0, -1.0])

dot1 = np.sum(x * y)
dot2 = x @ y

norm = np.sqrt(np.sum(x**2))
norm_np = np.linalg.norm(x)
```

The important conceptual point is that the index $i$ is summed over and therefore disappears. This is the simplest example of a contraction.

# Matrix-vector multiplication as index contraction

Matrix-vector multiplication can be written as

$$y_i = \sum_j A_{ij} x_j.$$

The index $j$ is contracted, while $i$ remains as a free index. This is why the output is a vector indexed by $i$.

```
A = np.random.rand(4, 3)
x = np.random.rand(3)

y = A @ x

A.shape   # (4, 3)
x.shape   # (3,)
y.shape   # (4,)
```

This interpretation is more general than the operation itself. It teaches us to read linear algebra as an operation on axes: some axes are matched and summed over, while others survive.

# Matrix-matrix multiplication

Matrix multiplication is another contraction:

$$C_{ik} = \sum_j A_{ij} B_{jk}.$$

The shared index $j$ is summed over. The free indices $i$ and $k$ define the shape of the output.

```
A = np.random.rand(4, 3)
B = np.random.rand(3, 5)

C = A @ B

C.shape   # (4, 5)
```

The operation is not merely “multiplying two rectangular arrays”. It has a precise index structure. Reversing the order changes the meaning and may even make the operation impossible.

```
# A @ B is valid: (4, 3) @ (3, 5) -> (4, 5)
# B @ A is invalid: (3, 5) @ (4, 3)
```

# Batched linear algebra

Many scientific computations involve repeating the same linear algebra operation for many independent systems, particles, parameter values, or time steps. These repetitions can often be represented using batch axes.

```
nbatch = 1000

A = np.random.rand(nbatch, 3, 3)
x = np.random.rand(nbatch, 3)

y = np.einsum("bij,bj->bi", A, x)

y.shape   # (nbatch, 3)
```

Here, each batch element contains one matrix-vector product. The batch index is not contracted; it labels independent problems.

→ **Example 01 - 02**

# Einstein summation notation in NumPy

`np.einsum` makes index structure explicit. It is useful when an operation is naturally written with indices, especially when the contraction is not a standard matrix product.

```
y = np.einsum("ij,j->i", A, x)
C = np.einsum("ij,jk->ik", A, B)

dot = np.einsum("i,i->", x, y)
```

The notation before -> describes input indices. The notation after -> describes output indices. Indices that appear in the inputs but not in the output are summed over.

A more physics-like example is a quadratic form:

```
energy = np.einsum("...i,ij,...j->...", v, M, v)
```

Here ... represents arbitrary leading batch dimensions.

# Tensor contractions beyond matrices

Higher-rank arrays often appear in mechanics, field theory, continuum models, machine learning, and numerical discretizations. Tensor contractions generalize matrix multiplication by summing over selected axes.

For example, suppose

$$w_i(x,y) = \sum_j \sigma_{ij}(x,y) v_j(x,y)$$

can be represented as:

```
sigma = np.zeros((nx, ny, 2, 2))
v = np.zeros((nx, ny, 2))

w = np.einsum("xyij,xyj->xyi", sigma, v)

w.shape   # (nx, ny, 2)
```

The spatial axes remain, the tensor component index $j$ is contracted, and the component index $i$ survives.

# Discrete derivatives as local array operations

Finite differences are local operations involving neighboring entries. In one dimension, the centered derivative and Laplacian can be written using slices.

```
du_dx = (u[2:] - u[:-2]) / (2*dx)

lap = (u[2:] - 2*u[1:-1] + u[:-2]) / dx**2
```

The output has a smaller shape because the boundary points were excluded. This is not only a coding detail; it reflects the fact that the stencil cannot be evaluated at the boundary without additional information.

For a two-dimensional field, the same idea extends axis by axis.

```
lap_inner = (
    (u[2:, 1:-1] - 2*u[1:-1, 1:-1] + u[:-2, 1:-1]) / dx**2
    +
    (u[1:-1, 2:] - 2*u[1:-1, 1:-1] + u[1:-1, :-2]) / dy**2
)
```

# Boundary conditions are part of the array model

Boundary conditions must be represented explicitly. They are not external to the numerical method; they determine how array entries near the edge are updated or interpreted.

For periodic boundaries, `np.roll` is often convenient:

```
lap_periodic = (
    np.roll(u, -1) - 2*u + np.roll(u, 1)
) / dx**2
```

For Dirichlet boundaries, one may update only the interior and keep boundary values fixed:

```
lap_inner = (u[2:] - 2*u[1:-1] + u[:-2]) / dx**2

rhs = np.zeros_like(u)
rhs[1:-1] = lap_inner + nonlinear_term(u[1:-1])
rhs[0] = 0.0
rhs[-1] = 0.0
```

Different boundary conditions produce different array operations and different physics.

# Residuals of differential equations

Many numerical methods require evaluating a residual: a discrete expression that should vanish for an exact solution. Residuals naturally have the same shape as the unknown field, at least on the interior grid.

For a schematic stationary nonlinear field equation,

$$-u_{xx} + u(u^2 - 1) = 0,$$

we might write:

```
lap = (u[2:] - 2*u[1:-1] + u[:-2]) / dx**2
res = -lap + u[1:-1] * (u[1:-1]**2 - 1.0)

res_norm = np.sqrt(np.mean(res**2))
```

The residual is an array-valued diagnostic. A scalar convergence measure is obtained only after a reduction.

→ **Example 01 - 03**

# Flattening and reshaping

Many solvers expect one-dimensional state vectors, even when the physical unknown is naturally a field. This creates a common need to move between structured arrays and flattened vectors.

```
u = np.zeros((nx, ny))

u_flat = u.ravel()
u_restored = u_flat.reshape((nx, ny))
```

This operation should be done carefully. Flattening removes explicit axis semantics from the shape, even though the information is still encoded in the ordering of entries. If we flatten a field, we must remember how to reconstruct its physical axes.

A typical ODE solver for a PDE after spatial discretization may require:

```
def rhs(t, y):
    u = y.reshape((nx, ny))
    dudt = compute_rhs_field(u)
    return dudt.ravel()
```

# Views, copies, and memory awareness

Some array operations create views of the same data, while others create copies. This matters for both correctness and performance.

```
u = np.arange(10.0)

v = u[2:7]        # usually a view
v[0] = -1.0

u                 # u has changed
```

In contrast, many arithmetic operations allocate new arrays:

```
w = 2.0*u + 1.0
```

For small arrays this is irrelevant. For large simulations, temporary arrays may dominate memory traffic and runtime. Efficient programming is therefore not only about avoiding Python loops, but also about understanding when data are copied and how much memory is moved.

# Vectorization and the performance model

Vectorization replaces many Python-level scalar operations by fewer array-level operations executed in optimized compiled code. This is why array-oriented code is usually much faster than explicit Python loops.

```
# Scalar-loop style
for i in range(n):
    y[i] = a*x[i] + b

# Array-oriented style
y = a*x + b
```

The second version expresses the operation at the level of the whole array. Internally, NumPy still loops over elements, but that loop is implemented in compiled code.

However, vectorization is not automatically optimal. An expression such as

```
y = a*x + b*z + c*np.sin(w)
```

may allocate temporary arrays. Later lectures will discuss when this matters and how tools such as Numba, Cython, JAX, or compiled kernels can help.

# Axis order and data layout

The order of axes affects both readability and performance. In NumPy's default C-style layout, the last axis is contiguous in memory. This means that operations along different axes may have different memory-access patterns.

```
u = np.zeros((nt, nx, ny))

u.strides
```

One common convention is to place batch or time axes first and component axes last:

```
field = np.zeros((nt, nx, ny))
vector_field = np.zeros((nt, nx, ny, 2))
```

This convention often reads naturally: first select time, then spatial location, then vector component. But other conventions may be preferable depending on external libraries, visualization tools, or linear algebra operations.

# Shape debugging as scientific debugging

Shape errors are often conceptual errors. They reveal that the code does not match the intended mathematical structure.

```
assert u.ndim == 3
assert u.shape == (nt, nx, ny)

mass = np.sum(u, axis=(1, 2)) * dx * dy
assert mass.shape == (nt,)
```

Printing shapes during development is not primitive; it is a legitimate way to verify the translation from mathematics to code.

```
print("u:", u.shape)
print("mean over space:", u.mean(axis=(1, 2)).shape)
print("centered:", (u - u.mean(axis=(1, 2), keepdims=True)).shape)
```

A wrong shape may indicate a wrong reduction, an unintended broadcast, an incorrectly placed batch axis, or a missing component dimension.

# Translating formulas into array-oriented computation

A practical translation strategy is to start from indices. Identify which indices are free, which are summed over, which represent spatial grids, which represent components, and which represent batches or ensembles. For example, suppose $u(t, x, y)$ is stored as:

```
# u: shape (nt, nx, ny)
```

Then a spatial integral as a function of time is:

```
mass_t = np.sum(u, axis=(1, 2)) * dx * dy
```

If $v_i(x, y, z)$ is a two-component vector field:

```
# v: shape (nt, nx, ny, 2)
speed = np.sqrt(np.sum(v**2, axis=-1))
```

The component axis is reduced, while time and space remain.

The general workflow is: assign meaning to axes, translate mathematical operations into elementwise operations, reductions, products, or contractions, and verify the resulting shape.

# A practical checklist

Useful questions:

- What does each axis represent?
- Which axes are preserved?
- Which axes are reduced?
- Which axes are contracted?
- Where are batch dimensions located?
- Which operations allocate new arrays?

These questions will guide later lectures on NumPy, efficient numerical kernels, profiling, compiled acceleration, parallelism, and tensor-based frameworks. Once the array structure is correct, optimization becomes a controlled process rather than a collection of tricks.

Section 4

# Further reading

# Supplementary literature


- Harris, C.R., Millman, K.J., van der Walt, S.J. *et al.* Array programming with NumPy. *Nature* **585**, 357–362 (2020). DOI: `https://doi.org/10.1038/s41586-020-2649-2`
- van der Walt, S., Colbert, S.C., Varoquaux, G. *The NumPy Array: A Structure for Efficient Numerical Computation. Computing in Science & Engineering* **13**(2), 22–30 (2011). DOI: `https://doi.org/10.1109/MCSE.2011.37`
- Trefethen, L.N., Bau, D. *Numerical Linear Algebra.* Society for Industrial and Applied Mathematics (SIAM), Philadelphia (1997). DOI: `https://doi.org/10.1137/1.9780898719574`
- LeVeque, R.J. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems.* Society for Industrial and Applied Mathematics (SIAM), Philadelphia (2007). DOI: `https://doi.org/10.1137/1.9780898717839`
- Kolda, T.G., Bader, B.W. *Tensor Decompositions and Applications. SIAM Review* **51**(3), 455–500 (2009). DOI: `https://doi.org/10.1137/07070111X`